

Relatório de Análise Crítica do Projeto

1. Informações do grupo

- **Curso:** Engenharia de Software
- **Disciplina:** Laboratório de Desenvolvimento de Software
- **Período:** 4º Período
- **Professor(a):** João Paulo Carneiro Aramuni
- **Membros do Grupo:** Davi Nunes, João Victor Russo

2. Identificação do Projeto

- **Nome do projeto:** Moeda Estudantil - Projeto de Software
- **Integrantes do outro grupo:** Karen Joilly, Jusue Carlos, Luiz Fernando
- **Link do repositório:**
<https://github.com/karenjoilly11/moeda-estudantil-projeto-de-software>

3. Arquitetura e Tecnologias Utilizadas

O projeto utiliza uma arquitetura baseada em microsserviços/módulos separados entre backend e frontend, promovendo modularidade e separação de responsabilidades.

Backend — Spring Boot

O backend foi desenvolvido utilizando **Spring Boot (Java)**, seguindo uma arquitetura em camadas bem definidas:

- **Controllers:** (ex: `AlunoController`) recebem requisições HTTP REST.
- **Services:** (ex: `AlunoService`, `TransacaoService`) concentram as regras de negócio complexas.
- **Repositories:** utilizam Spring Data JPA para acesso aos dados.
- **Eventos Assíncronos:** O projeto utiliza o `ApplicationEventPublisher` (com indicativos de uso de mensageria via RabbitMQ) para publicação de eventos, como o envio de e-mails (`EmailResgateAlunoEvent`).



Frontend

O frontend está isolado em um diretório próprio, utilizando **Node.js** e um framework reativo (React/Next.js). A comunicação com o backend ocorre exclusivamente via APIs REST.



Integração e Segurança

- A autenticação é baseada em Tokens (JWT ou similar), manipulados pela classe utilitária `TokenUtil`.
- O armazenamento de senhas utiliza `PasswordEncoder` (BCrypt) para garantir a segurança dos dados sensíveis no banco.

4. Organização do GitHub e Fluxo de Trabalho Colaborativo

4.1. Estrutura do Repositório

A separação estrutural é clara: pastas `backend` e `frontend` isolam as bases de código, facilitando a containerização (Docker) e a alocação de equipes.

4.2. Padrões de Código

O projeto backend segue as convenções padrão do Spring (pacotes `controller`, `services`, `dto`, `entities`, `repositories`). O uso de anotações do Lombok (`@RequiredArgsConstructor`) reduz a verbosidade do código, indicando maturidade na stack Java.

4.3. Oportunidades de Melhoria

Apesar da boa organização de pastas, o fluxo de *branching* e uso rigoroso de PRs (com *Code Review*) precisa ser validado. A adoção de *Conventional Commits* ajudaria a automatizar a geração de changelogs e rastreabilidade das tarefas.

5. Dificuldade para Configuração do Ambiente

(Adapte esta seção de acordo com a experiência real que vocês tiveram rodando o projeto)

5.1. Requisitos de Ferramentas

O projeto demanda a instalação de múltiplas ferramentas: Java (JDK 17+), Maven para o backend, Node.js para o frontend, e potencialmente um broker de mensageria (RabbitMQ) e banco de dados (PostgreSQL/H2). A orquestração via Docker (`docker-compose`) amenizaria esses atritos se bem documentada.

5.2. Aspectos a Analisar e Soluções Aplicadas

- **Variáveis de Ambiente:** É necessário configurar adequadamente credenciais de banco de dados e segredos de JWT na inicialização do Spring.
- **Serviço de Email:** Como o `TransacaoService` dispara eventos de e-mail, foi necessário configurar ou *mockar* esse serviço para testar o fluxo de resgate sem erros locais.

6. Análise de Qualidade do Código e Testes

6.1. Design e Princípios SOLID

- **Acoplamento e Coesão:** As responsabilidades estão bem divididas entre Controllers e Services. No entanto, algumas classes de serviço (como `TransacaoService`) estão assumindo múltiplas responsabilidades em métodos longos (violação do *Single Responsibility Principle*).
- **Code Smells:**
 - **Duplicated Code:** Notado no `AlunoController`, onde a verificação de autorização do usuário via token é repetida nos métodos `atualizarPerfilComId` e `excluirConta`.
 - **Long Method:** O método `resgatar` em `TransacaoService` possui mais de 100 linhas e lida com validações, regras financeiras, persistência e mensageria simultaneamente.

6.2. Segurança e Tratamento de Erros

- **Pronto Positivo:** O uso de `PasswordEncoder` para armazenar senhas criptografadas e a verificação de permissões conferindo o ID extraído do Token (`!id.equals(alunoldAutenticado)`).
- **Ponto de Melhoria:** Exceções genéricas como `throw new RuntimeException(...)` estão sendo lançadas na camada de Service. O ideal seria criar exceções personalizadas (ex: `EmailJaCadastradoException`, `SaldoInsuficienteException`) e usar um `@ControllerAdvice` para mapeá-las para os Status HTTP corretos.



7. Sugestões de Melhorias

1. **Tratamento Global de Exceções:** Implementar um `@RestControllerAdvice` para padronizar os retornos de erro da API e evitar o uso de `RuntimeException` pura com verificações de string baseadas em `e.getMostSpecificCause().getMessage().contains("cpf")`.
2. **Uso de Enums:** Substituir strings fixas mágicas (como "PENDENTE" e "UTILIZADO" no `TransacaoService`) por `Enums` (`StatusTransacao`), garantindo tipo seguro e evitando erros de digitação.
3. **Padrão DTO-Entity:** Adotar bibliotecas como `MapStruct` ou `ModelMapper` para remover a lógica manual e repetitiva de conversão entre DTOs e Entidades nos Services.
4. **Isolamento de Componentes (SOLID):** O `TransacaoService` não deveria se preocupar em instanciar os eventos de e-mail de forma *hardcoded*. O ideal seria extrair isso para uma classe publicadora especializada.
5. **Interceptadores para Autorização:** Centralizar a verificação de posse do recurso (verificar se o token pertence ao ID requisitado) em filtros ou anotações do `Spring Security` (como `@PreAuthorize`), limpando os Controllers.



8. Refatorações Propostas (3 partes do código)

Refatoração 1 – Remoção de Código Duplicado

Arquivo: `backend/src/main/java/com/moedaestudantil/api/controller/AlunoController.java`

Tipo de refatoração aplicada: Replace Duplicated Code with Method (Extração de lógica de validação de permissão)

Pull Request: [Cole o Link Aqui]



Antes

Java

```
@PostMapping("/perfil/{id}")
public ResponseEntity<?> atualizarPerfilComId(
    @RequestBody AlunoPerfilDTO dto,
    @PathVariable Long id,
    @RequestHeader(value = "Authorization", required = false) String authorization) {
    Long alunoldAutenticado = extractAlunoldFromToken(authorization);
    if (!id.equals(alunoldAutenticado)) {
        throw new SecurityException("Você não tem permissão para editar este perfil");
    }
}
```

```

    }
    AlunoResponseDTO resultado = alunoService.atualizarPerfil(id, dto);
    return ResponseEntity.ok(resultado);
}

@DeleteMapping("/{id}")
public ResponseEntity<?> excluirConta(
    @PathVariable Long id,
    @RequestHeader(value = "Authorization", required = false) String authorization) {
    Long alunoldAutenticado = extractAlunoldFromToken(authorization);
    if (!id.equals(alunoldAutenticado)) {
        throw new SecurityException("Você não tem permissão para excluir esta conta");
    }
    alunoService.excluirConta(id);
    return ResponseEntity.ok().body("Conta excluída com sucesso");
}

```

● Depois

Java

```

private void validarPermissaoAcesso(Long idRequisitado, String authorization, String
acao) {
    Long alunoldAutenticado = extractAlunoldFromToken(authorization);
    if (!idRequisitado.equals(alunoldAutenticado)) {
        throw new SecurityException("Você não tem permissão para " + acao);
    }
}

```

```

@PutMapping("/perfil/{id}")
public ResponseEntity<?> atualizarPerfilComId(
    @RequestBody AlunoPerfilDTO dto,
    @PathVariable Long id,
    @RequestHeader(value = "Authorization", required = false) String authorization) {
    validarPermissaoAcesso(id, authorization, "editar este perfil");
    AlunoResponseDTO resultado = alunoService.atualizarPerfil(id, dto);
    return ResponseEntity.ok(resultado);
}

```

```

@DeleteMapping("/{id}")
public ResponseEntity<?> excluirConta(
    @PathVariable Long id,
    @RequestHeader(value = "Authorization", required = false) String authorization) {
    validarPermissaoAcesso(id, authorization, "excluir esta conta");
    alunoService.excluirConta(id);
    return ResponseEntity.ok().body("Conta excluída com sucesso");
}

```

Justificativa

A repetição da lógica de validação de identidade fere o princípio DRY (*Don't Repeat Yourself*). A extração para o método `validarPermissaoAcesso` simplifica a leitura dos endpoints e centraliza a regra de segurança, facilitando manutenções futuras.

Refatoração 2 – Extração de Método (Extract Method) para SRP

Arquivo: `backend/src/main/java/com/moedaestudantil/api/services/TransacaoService.java`

Tipo de refatoração aplicada: Extract Method (Separação do fluxo de Validação)

Pull Request: [Cole o Link Aqui]

● Antes

Java

```
@Transactional
public ResgateResponseDTO resgatar(ResgateRequestDTO dto, Long
alunoldAutenticado) {
    Objects.requireNonNull(alunoldAutenticado, "alunoldAutenticado obrigatório");
    if (!alunoldAutenticado.equals(dto.getAlunold())) {
        throw new SecurityException("Não é permitido resgatar em nome de outro aluno");
    }

    Aluno aluno = alunoRepository.findById(dto.getAlunold())
        .orElseThrow(() -> new EntityNotFoundException("Aluno não encontrado!"));
    Vantagem vantagem = vantagemRepository.findById(dto.getVantagemId())
        .orElseThrow(() -> new EntityNotFoundException("Vantagem não encontrada!"));

    // validar saldo
    if (aluno.getSaldoMoedas() < vantagem.getCustoMoedas()) {
        throw new IllegalStateException("Saldo insuficiente...");
    }

    // validar estoque
    if (vantagem.getEstoque() != null && vantagem.getEstoque() <= 0) {
        throw new IllegalStateException("Vantagem sem estoque");
    }

    // ... continua com atualizar saldo, atualizar estoque, gerar cupom, etc.
```

● Depois

Java

```
private void validarRegrasDeResgate(Aluno aluno, Vantagem vantagem) {
    if (aluno.getSaldoMoedas() < vantagem.getCustoMoedas()) {
        throw new IllegalStateException("Saldo insuficiente. Saldo atual: "
            + aluno.getSaldoMoedas() + " | Custo: " + vantagem.getCustoMoedas());
    }
    if (vantagem.getEstoque() != null && vantagem.getEstoque() <= 0) {
        throw new IllegalStateException("Vantagem sem estoque");
    }
}
```

```

}

@Transactional
public ResgateResponseDTO resgatar(ResgateRequestDTO dto, Long
alunoldAutenticado) {
    Objects.requireNonNull(alunoldAutenticado, "alunoldAutenticado obrigatório");
    if (!alunoldAutenticado.equals(dto.getAlunold())) {
        throw new SecurityException("Não é permitido resgatar em nome de outro aluno");
    }

    Aluno aluno = alunoRepository.findById(dto.getAlunold())
        .orElseThrow(() -> new EntityNotFoundException("Aluno não encontrado!"));
    Vantagem vantagem = vantagemRepository.findById(dto.getVantagemId())
        .orElseThrow(() -> new EntityNotFoundException("Vantagem não encontrada!"));

    validarRegrasDeResgate(aluno, vantagem);

    // ... continua com a lógica de atualização e criação de transação
}

```

Justificativa

O método principal estava acumulando responsabilidades demais (code smell "Long Method"). Extraíndo as regras de negócio para **validarRegrasDeResgate**, o código fica modular e altamente testável via testes unitários isolados, aderindo melhor ao *Single Responsibility Principle* (SRP).

Refatoração 3 – Extração de Mapeamento de Entidades

Arquivo: [backend/src/main/java/com/moedaestudantil/api/services/AlunoService.java](#)

Tipo de refatoração aplicada: Extract Method (Mapeamento DTO -> Entidade)

Pull Request: [\[Cole o Link Aqui\]](#)

● Antes

Java

```

public AlunoResponseDTO cadastrar(AlunoCadastroDTO dto) {
    if (alunoRepository.existsByEmail(dto.getEmail())) {
        throw new RuntimeException("Email já cadastrado!");
    }

    Instituicao instituicao = instituicaoRepository.findById(dto.getInstituicaold())
        .orElseThrow(() -> new RuntimeException("Instituição não encontrada!"));

    Aluno aluno = new Aluno();
    aluno.setNome(dto.getNome());
    aluno.setEmail(dto.getEmail());
    aluno.setSenha(passwordEncoder.encode(dto.getSenha()));
    aluno.setTipo(TipoUsuario.ALUNO);
    aluno.setInstituicao(instituicao);
}

```

```

aluno.setCpf(dto.getCpf());
aluno.setRg(dto.getRg());
aluno.setEndereco(dto.getEndereco());
aluno.setCurso(dto.getCurso());
aluno.setSaldoMoedas(0.0);

try {
    Aluno salvo = alunoRepository.save(aluno);
    return toResponseDTO(salvo);
} catch (DataIntegrityViolationException e) {
    // tratamento
}
}

```

● Depois

Java

```

private Aluno mapearParaEntidade(AlunoCadastroDTO dto, Instituicao instituicao) {
    Aluno aluno = new Aluno();
    aluno.setNome(dto.getNome());
    aluno.setEmail(dto.getEmail());
    aluno.setSenha(passwordEncoder.encode(dto.getSenha()));
    aluno.setTipo(TipoUsuario.ALUNO);
    aluno.setInstituicao(instituicao);
    aluno.setCpf(dto.getCpf());
    aluno.setRg(dto.getRg());
    aluno.setEndereco(dto.getEndereco());
    aluno.setCurso(dto.getCurso());
    aluno.setSaldoMoedas(0.0);
    return aluno;
}

```

```

public AlunoResponseDTO cadastrar(AlunoCadastroDTO dto) {
    if (alunoRepository.existsByEmail(dto.getEmail())) {
        throw new RuntimeException("Email já cadastrado!");
    }
}

```

```

Instituicao instituicao = instituicaoRepository.findById(dto.getInstituicaoId())
    .orElseThrow(() -> new RuntimeException("Instituição não encontrada!"));

```

```

Aluno aluno = mapearParaEntidade(dto, instituicao);

```

```

try {
    Aluno salvo = alunoRepository.save(aluno);
    return toResponseDTO(salvo);
} catch (DataIntegrityViolationException e) {
    // tratamento
}

```

}

Justificativa

A construção e preenchimento de objetos a partir de DTOs é puramente estrutural e polui a leitura do fluxo de negócio do método principal (cadastro, verificações, save e exception handling). Isolar o builder no método auxiliar `mapearParaEntidade` torna o método `cadastrar` muito mais descritivo e coeso.

9. Conclusão

A análise crítica permitiu identificar aspectos importantes relacionados à organização geral do projeto Moeda Estudantil. A investigação detalhada evidenciou pontos positivos, como a separação Fullstack (Spring Boot + Web Frontend) e boas escolhas tecnológicas como JWT e mensageria. No entanto, expôs pequenos *code smells* de duplicidade e centralização de regras que atrapalham a testabilidade.

As refatorações propostas tiveram impacto direto na **melhoria da legibilidade** e na **clareza das responsabilidades**. Com as modificações enviadas nos Pull Requests, a camada lógica ficará mais polida, ajudando o grupo detentor do repositório a sustentar e escalar o software futuramente.